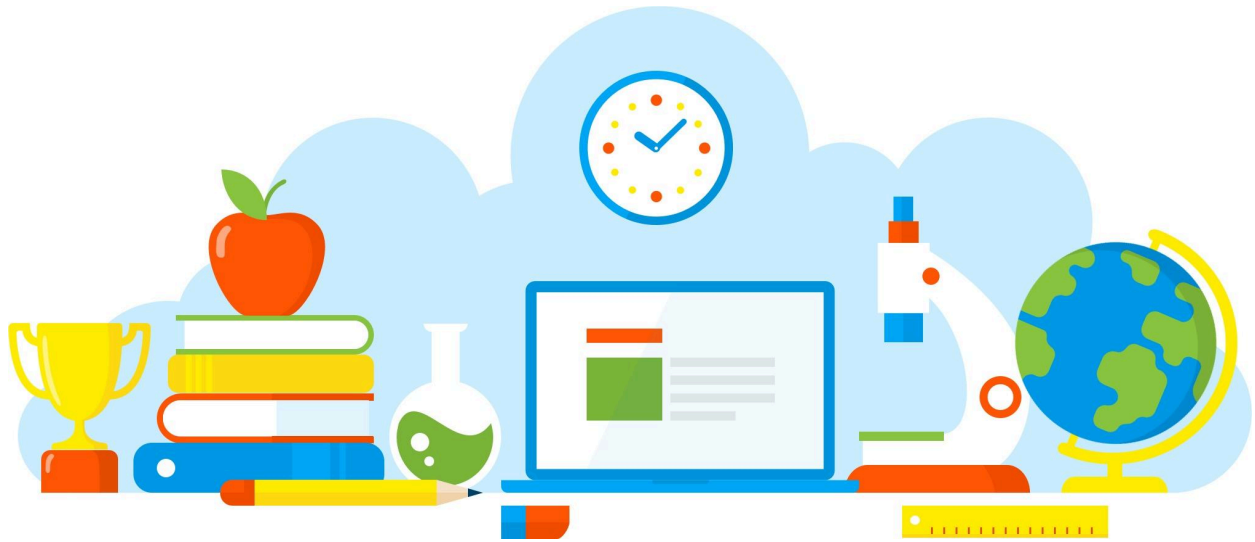




Projet crypto-bot

Julien CAILLABET et Maxime MANGA
21/07/2026

Lexique	3
Introduction	4
Structure du projet	5
Remarques générales	6
Réseaux	6
Sous-fichiers docker-compose	6
Volumes	7
Définition des conteneurs	7
FastAPI	7
PostgreSQL	8
Test unitaires	9
Streamlit	10
Prometheus/Grafana	10
Airflow	11
Cassandra	11
FastAPI	13
Design pattern MVC	13
Fichier principal	14
Routes	15
Ingestion	16
CCXT	16
Contraintes	17
Routes	18
Coingecko	18
Base(s) de données	21
PostgreSQL	22
Cassandra	23
Modélisation	23
Test de performance	24
Partitionnement	25
Manipulation	26
Routes	27
Machine learning	28
Difficultés rencontrées	28
Prédiction par classification	28
Transformation des données	30
Routes FastAPI	31
Pipeline Airflow	32
Choix techniques	33
Dynamic Task Mapping	33

Task group	33
Duplication de tâches	34
Appel manuel	34
DAGs	34
Task groups	34
coin_toggle_dag	36
ingestion_dag	36
streaming_dag	37
Ce dernier pipeline est exécuté chaque heure.	37
Tests unitaires	38
Monitoring	39
FastAPI	39
Logs	39
CI/CD	41
Test unitaire	41
Image docker	42
Dashboard	43
Administration	43
Consultation	44
Conclusion / Piste d'amélioration	45
Sources	46

Lexique

- **Bucket date** : date faisant partie de la clé de partitionnement de la table de stockage des bougies dans la base de données Cassandra. Cette date sera de la forme "début de mois", "début d'année" ... suivant le pas de récupération de données choisi.
- **Catalogue** : Liste des crypto-monnaies que le projet est capable de gérer.
- **CCXT** : API utilisée pour récupérer le cours des crypto-monnaies.
- **Coin** : Terme anglais pour parler d'une crypto-monnaie, c'est également le terme technique utilisé dans le code.
- **DAG** : nom donné à un pipeline de traitement de données réalisé avec Airflow.
- **Dynamic Task Mapping** : fonctionnalité de Airflow pour avoir plusieurs tâches en parallèle, sans connaître à l'avance le nombre de ces tâches.
- **Exchange** : Service permettant de récupérer le cours d'une crypto-monnaie, c'est également le terme technique utilisé dans le code.
- **Healthcheck** : commande définie dans docker-compose pour déterminer si un conteneur est bien initialisé et est prêt à être utilisé.
- **Landing zone** : Zone recevant les différents fichiers générés par notre API, et nécessaires à plusieurs tâches. Dans notre projet, elle est simulée par la présence d'un simple dossier hors du conteneur de cette API.
- **Lifespan** : zone de code exécutée avant le lancement de l'application FastAPI. Cela permet par exemple d'initialiser une connexion de base de donnée accessible pendant tout le cycle de vie de l'application.
- **Réseau** : réseau défini au niveau de docker. Un conteneur ne peut communiquer qu'avec ceux faisant partie du même réseau.
- **Streamlit** : technologie utilisée pour réaliser les dashboard de notre projet
- **Task group** : regroupement de tâches devant être utilisées à plusieurs endroits dans les pipelines Airflow
- **Technologie** : Terme utilisé pour parler d'un service à déployer dans docker (FastAPI , bases de données ...), pour ne pas faire doublon avec la notion de service du pattern MVC

Introduction

L'objectif du projet à été de réaliser un bot de trading pour le marché des crypto-monnaies.

Nous avons choisi de nous mettre à la place d'un utilisateur d'une hypothétique application de trading pour dicter les différents choix techniques de notre architecture. Dans nos dashboard de présentation du projet, nous souhaitons pouvoir simuler des achats et des reventes de crypto-monnaies.

Nous avons supposé qu'une telle application devrait permettre de choisir sur quelles crypto-monnaies pouvoir réaliser des échanges, et de pouvoir avoir un affichage temps-réel¹ de l'évolution du cours pour une monnaie donnée, et de même pour l'affichage de la prédiction.

Il en ressort 3 besoins essentiels dans ce projet :

- Ne pas être bloqué sur une seule monnaie et pouvoir être capable d'en gérer plusieurs de façon générique (sans être bloqué sur une seule API).
- Être capable d'afficher en temps réel les données (cours + prédiction).
- Implémenter le pipeline qui fera la liaison entre les deux partie précédentes.

¹ Il sera présenté plus tard que nous n'avons pas de temps réel véritable, à cause de la configuration que nous dictera l'architecture du machine learning, même si il sera possible de le faire.

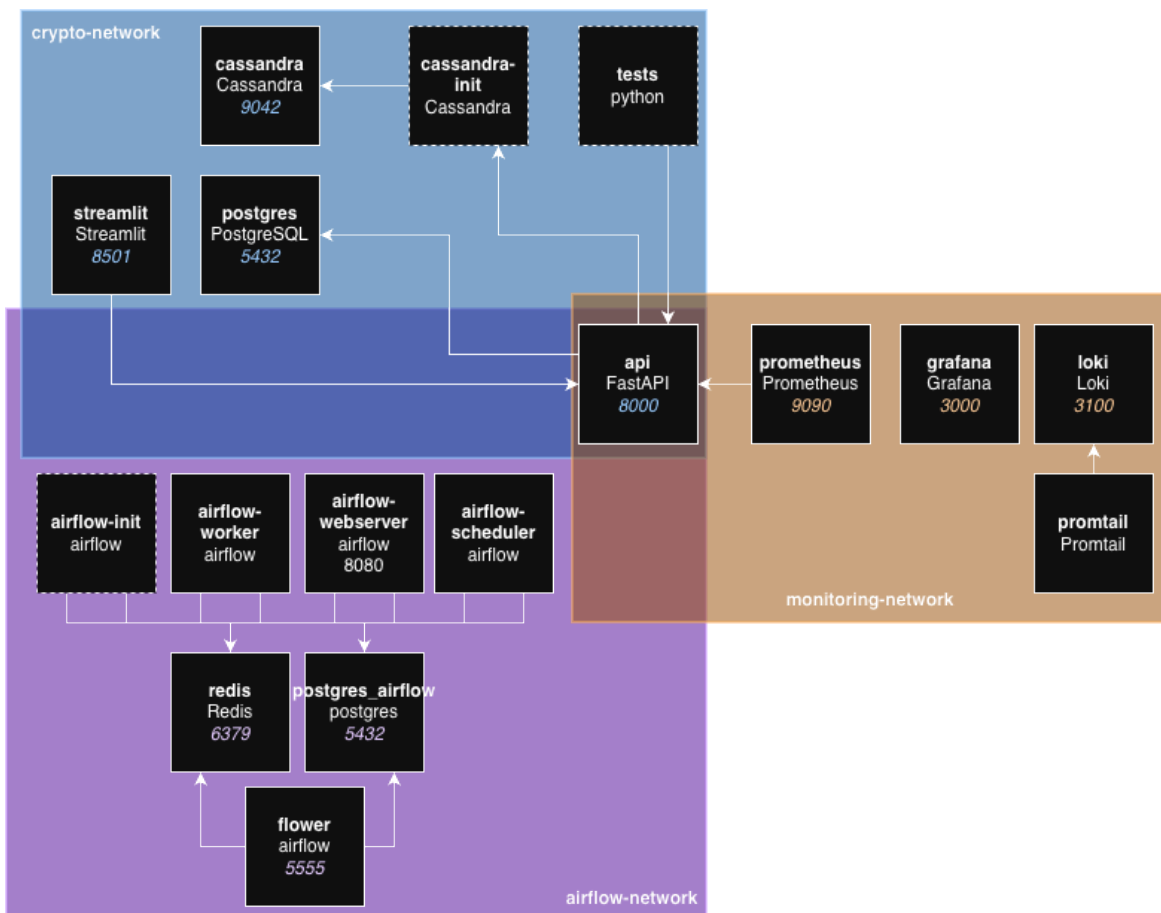
Structure du projet

La première priorité du projet a été la mise en place d'un environnement de développement local à la fois robuste pour que chaque développeur puisse travailler sans aucun paramétrage, et facilement adaptable pour une mise en production hypothétique.

C'est pour cette raison que nous avons choisi docker-compose pour cette structuration. Ainsi, chaque technologie utilisée est encapsulée dans son propre conteneur docker, de façon automatique de sorte de n'avoir rien à installer. Une fois le code récupéré depuis notre repo git (<https://github.com/jcaillabet-ia/MAI26-BDE-OPA-2>), notre environnement local peut être lancé grâce à une simple commande :

docker-compose up

La structure générale de notre environnement docker-compose peut être représentée par le schéma suivant :



Remarques générales

Chaque conteneur sera détaillé plus loin, mais plusieurs points généraux peuvent être abordés

Réseaux

Notre architecture est répartie dans trois réseaux docker (un conteneur docker ne peut communiquer qu'avec ceux faisant partie du même réseau) :

- crypto-network : conteneurs nécessaires à l'architecture de notre API
- airflow-network : conteneurs nécessaires au fonctionnement de Airflow
- monitoring-network : conteneurs nécessaires aux fonctionnalités de monitoring

Dans docker-compose, les réseaux sont définis de cette façon :

```
networks:
  airflow-network:
  crypto-network:
  monitoring-network:
```

Sous-fichiers docker-compose

Par souci de lisibilité, si un outil a besoin de plusieurs services, il sera renseigné dans son propre sous-fichier **.yaml**. Deux technologies sont concernées par ce choix :

- Airflow
- Cassandra

Dans docker-compose, cela se définit de cette façon :

```
include:
  - docker-compose.airflow.yaml
  - docker-compose.cassandra.yaml
```

Volumes

Pour chaque type de base de données utilisé par notre projet, nous avons défini un volume afin d'assurer la persistance des données entre chaque lancement de notre environnement :

- postgres_data : persistance de la base de données PostgreSQL
- cassandra_data : persistance de la base de données Cassandra

Dans docker-compose cela se définit de cette façon :

```
volumes:  
  postgres_data:  
  cassandra_data:
```

Définition des conteneurs

FastAPI

FastAPI est lancé sur le port 8000, et est déclaré de cette façon :

```
api:  
  build:  
    context: ./api  
    dockerfile: Dockerfile  
  restart: always  
  container_name: api  
  ports:  
    - "8000:8000"  
  volumes:  
    - ./api:/app  
    - ./data:/app/data  
  environment:  
    - DATABASE_URL=postgresql://postgres:postgres@postgres:5432/postgres  
    - PYTHONPATH=/app:/app/src  
  depends_on:  
    cassandra-init:  
      condition: service_completed_successfully  
    postgres:  
      condition: service_healthy  
  networks:  
    airflow-network: {}  
    crypto-network: {}  
    monitoring-network:  
      aliases:  
        - api-monitoring  
  healthcheck:  
    test: [ "CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/docs')" ]  
    interval: 10s  
    timeout: 5s  
    retries: 6
```

Remarques :

- En plus de son propre volume, il utilise un deuxième volume (appelé **data**), pour simuler l'existence d'une "landing zone" dans notre projet.
- Le service FastAPI se lancera qu'à partir du moment ou le service postgres sera lancé, et aussi après que le service d'initialisation de Cassandra (cassandra-init) soit terminé.
- Le service FastAPI possède une règle healthcheck qui permet de vérifier s' il a terminé son démarrage (nécessaire par exemple pour lancer les tests unitaires).

PostgreSQL

PostgreSQL est lancé sur le port 5432, et est déclaré de cette façon :

```
postgres:
  image: postgres:15-alpine
  restart: always
  container_name: postgres
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: postgres
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./api/scripts/schema_postgres.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - crypto-network
```

Remarques:

- En plus de son propre volume de persistance des données, il utilise un deuxième volume pour fournir à postgres un fichier d'initialisation des données. Ce qui permet d'utiliser directement notre projet sans avoir à l'initialiser manuellement.

- Le service postgres possède un règle healthcheck qui permet de vérifier si il a terminé son démarrage (nécessaire pour lancer le conteneur FastAPI).

Test unitaires

Les tests unitaires sont effectués dans un simple conteneur python, en utilisant la librairie pytest. Il est déclaré de cette façon :

```
tests:
  image: python:3.11-slim
  container_name: tests
  profiles:
    - test
  volumes:
    - ./tests:/app
  working_dir: /app
  environment:
    - PYTHONPATH=/app
  depends_on:
    api:
      condition: service_healthy
  entrypoint: >
    sh -c " pip install httpx && pip install pytest && pytest -v "
  networks:
    crypto-network:
```

Remarques:

- Les tests unitaires sont gérés avec un simple service python, qui doit se lancer manuellement si on souhaite lancer les tests en local (définir un profile docker-compose permet de l'exclure du lancement de l'exécution générale de notre projet). Ce même service sera lancé automatiquement dans les pipelines Github Actions.
- Ce service attend que le service FastAPI soit bien lancé pour pouvoir effectuer les tests unitaires.

Streamlit

Streamlit est lancé sur le port 8501, et est déclaré de cette façon :

```
streamlit:
  build: ./streamlit
  command: streamlit run streamlit/main.py --server.port 8501
  volumes:
    - ./usr/src/app
  ports:
    - 8501:8501
  image: yourstreamlitapp:latest
  depends_on:
    api:
      condition: service_healthy
  networks:
    crypto-network:
```

Remarques:

- Ce conteneur attend que le service FastAPI soit bien opérationnel avant de se lancer

Prometheus/Grafana

Les conteneurs prometheus et grafana sont lancés respectivement sur les ports 9090 et 3000, et sont déclarés de cette façon :

```
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports:
    - "9090:9090"
  networks:
    - monitoring-network
  depends_on:
    api:
      condition: service_healthy
  > Run Service
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
  logging:
    driver: "none"
  volumes:
    - ./grafana/provisioning:/etc/grafana/provisioning:ro
    - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
  networks:
    - monitoring-network
```

Remarques:

- Prometheus attend que le service FastAPI soit bien opérationnel avant de pouvoir se lancer
- Le service prometheus définit un volume pour lui fournir sa configuration (liaison à FastAPI)
- Le service grafana définit deux volumes pour lui fournir sa configuration (dashboards par défaut par exemple)

Airflow

Par soucis de lisibilité, le contenu du fichier docker-compose.yml spécifique à Airflow ne sera pas détaillé dans ce rapport (nous avons utilisé le même fichier qu l'exemple vu en cours). Cependant, nous y avons apporté une modification :

- Nous avons défini cette variable d'environnement

```
AIRFLOW__CORE__LOAD_EXAMPLES: "false"
```

Pour ne pas avoir les DAG d'exemple affichés dans notre interface Airflow.

Cassandra

Cassandra est intégré dans notre projet en utilisant deux conteneurs, définis de cette façon :

```
cassandra:
  image: cassandra:4
  container_name: cassandra
  restart: always
  environment:
    - MAX_HEAP_SIZE=1G
    - HEAP_NEWSIZE=256M
    - CASSANDRA_RPC_ADDRESS=0.0.0.0
    - CASSANDRA_BROADCAST_RPC_ADDRESS=cassandra
  ports:
    - "9042:9042"
  volumes:
    - cassandra_data:/var/lib/cassandra
  healthcheck:
    test: [ "CMD-SHELL", "nodetool status | grep -q 'UN'" ]
    interval: 15s
    timeout: 5s
    retries: 10
  networks:
    - crypto-network

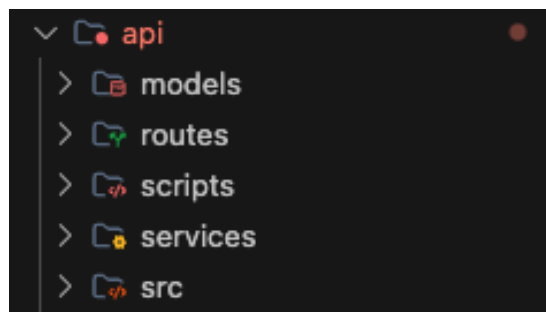
# Run Service
cassandra-init:
  image: cassandra:latest
  container_name: cassandra-init
  depends_on:
    cassandra:
      condition: service_healthy
  volumes:
    - ./api/scripts/schema_cassandra.cql:/init.cql
  entrypoint: >
    bash -c "echo 'Cassandra prête, initialisation du schéma...'; cqlsh cassandra 9042 -f /init.cql && echo 'Schéma initialisé avec succès.'"
  networks:
    - crypto-network
```

-
- Un conteneur pour exécuter cassandra, utilisant le port 9042
 - Un service pour initialiser cassandra (contrairement à PostgreSQL, Cassandra ne permet pas de définir un script d'initialisation) :
 - Il attend que cassandra soit bien lancé pour pouvoir faire l'initialisation
 - L'initialisation est faite à chaque lancement de l'environnement. Toutes les requêtes d'initialisation intègrent donc les mots clés **IF NOT EXISTS** pour chaque élément à créer .

FastAPI

Design pattern MVC

Pour structurer le code de l'API FastAPI, élément central de notre projet, nous avons choisi de suivre le design pattern MVC (de façon adaptée) pour structurer l'arborescence des fichiers. Cela nous permet une meilleure maintenabilité du code et une plus grande lisibilité de la structure des fichiers :



On y retrouve :

- models : classes de mapping de tables de base de données (utilisé principalement pour PostgreSQL)
- routes : fichiers de déclaration de routes (chaque fichier gère une entité/concept spécifique)
- script : fichiers de scripts pour manipuler les base de données
- services : fonctions utilitaires utilisables principalement dans les routes. Sur le même principe, chaque service gère une seule entité/concept
- src : classes python utilisées dans les différents services

Concernant les entité/concept gérés dans l'API, on peut lister :

- admin : routes d'administration (vidage de tables, réinitialisations des données ...), que nous pouvons appeler manuellement, pour nous faciliter certains tests.
- candle : gestion des bougies (qui décrivent les données que nous manipulons)
- coin : gestion du catalogue de crypto-monnaies
- database : gestion des base de données (enregistrement/chargement)
- ingestion : récupération des données depuis CCXT
- ml : gestion du machine learning

Fichier principal

Grâce à la mise en place du pattern MVC, nous nous retrouvons avec un fichier main.py minimal pour FastAPI :

```
from cassandra.cluster import Cluster
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from prometheus_fastapi_instrumentator import Instrumentator

from routes import admin as admin_router
from routes import candle as candle_router
from routes import coin as coin_router
from routes import ingestion as ingestion_router
from routes import ml as ml_router

@asynccontextmanager
async def lifespan(app: FastAPI):
    CLUSTER_IPS = ['cassandra']
    keyspace = "crypto_bot"
    cluster = Cluster(CLUSTER_IPS, compression=True)
    session = cluster.connect(keyspace=keyspace)
    app.state.cassandra_session = session
    yield
    cluster.shutdown()

app = FastAPI(title="API FastAPI du projet crypto-bot", lifespan=lifespan)
Instrumentator().instrument(app).expose(app)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(admin_router.router, prefix="/admin", tags=["admin"])
app.include_router(candle_router.router, prefix="/candle", tags=["candle"])
app.include_router(coin_router.router, prefix="/coin", tags=["coin"])
app.include_router(ingestion_router.router, prefix="/ingestion", tags=["ingestion"])
app.include_router(ml_router.router, prefix="/ml", tags=["ml"])
```

En dehors des imports classiques, on retrouve les éléments suivants dans notre fichier principal :

- définition d'un "lifespan" FastAPI pour initialiser le connecteur vers la base cassandra (pour économiser quelques millisecondes à chaque requête, le temps d'accès à cassandra étant critique dans notre pipeline, par rapport à l'exigence de temps-réel)
- initialisation de FastAPI, avec l'utilisation de la classe "Instrumentator", permettant la connexion entre FastAPI et Prometheus

Routes

default ^ GET /metrics Metrics	coin ^ GET /coin/ List GET /coin/{id} Item PATCH /coin/{id}/enable Enable PATCH /coin/{id}/disable Disable
admin ^ POST /admin/postgres/empty Empty Postgres POST /admin/cassandra/empty Empty Cassandra POST /admin/postgres/fill Fill Postgres	ingestion ^ POST /ingestion/run Run Ingestion GET /ingestion/stream/{coin_id} Stream Candle
candle ^ GET /candle/{id}/list List POST /candle/{id}/save Save GET /candle/{coin_id}/interval Last Date DELETE /candle/{id}/remove Remove	ml ^ GET /ml/transform/{coin_id} Transform POST /ml/train Train POST /ml/predict Predict

On retrouve bien la séparation des entités/concepts que nous avons choisie pour structurer le service FastAPI. (la nature des routes sera détaillées dans chaque partie correspondante)

La routes **/metrics** est une route ajoutée automatiquement par la classe Instrumentator pour rendre accessible les metrics de monitoring de FastAPI à Prometheus.

Ingestion

Dans le cadre de notre projet, l'étape d'ingestion consistera à récupérer le cours des différentes crypto-monnaies à gérer. Comme pour les cours de la bourse, celui des crypto-monnaies se récupère sous la forme de "bougies" (données k-line). Elles comportent généralement les informations suivantes, pour la période de récupération choisie (minute, heure ...) :

- open
- high
- low
- close
- volume

On peut appeler cela aussi format "ohlcv".

Sur un plan plus technique, nous avons choisi d'utiliser deux APIs dans la phase d'ingestion, chacune ayant une responsabilité bien précise.

CCXT

CCXT est une librairie qui permet de récupérer les données de différentes crypto-monnaies, à travers la connexion à plus d'une centaine de "exchanges" (marchés), chaque exchange étant capable de gérer un certain nombre de monnaies, une monnaie n'étant pas forcément disponible chez tous les exchanges.

Elle fournit une API permettant de consulter les différentes sources de données de manière unifiée, pour ne pas avoir à gérer les spécificités de chacune côté utilisateur. Par exemple, on récupère les bougies des cryptomonnaies grâce à une fonction **fetch_ohlcv**, quel que soit l'échange qu'on souhaite consulter.

CCXT sera utilisé de deux façons différentes :

- Consultation classique de l'API pour récupérer l'historique des données passées
- Consultation par websocket pour récupérer les données en temps réel, dans l'optique de faire du streaming dans des versions futures du projet (qui récupéreront par exemple les données avec des pas à la seconde).

Contraintes

Dans notre projet, comme nous souhaitons de base être capable de gérer autant de crypto-monnaies que l'on souhaite, il peut être possible qu'entre une certaine monnaie et une autre on doit utiliser des échanges différents. Il faut donc être capable de récupérer dynamiquement l'échange qu'on doit utiliser à partir de son prénom (par exemple pour binance l'appel classique se fait avec le code `ex = ccxt.binance()`).

Ce besoin de réflexivité peut être comblé en python de la façon suivante :

```
exchange_id = "binance"  
exchange_class = getattr(ccxt, exchange_id, None)
```

Pour pouvoir entraîner notre modèle de machine learning, nous allons avoir besoin de beaucoup de données. Comme la plupart des échanges sont limités en nombre de points requêttables, nous devons mettre en place une pagination de la récupération des données. A chaque demande de l'historique d'une monnaie, nous devons donc lire plusieurs paquets, puis fusionner les différents résultats.

Nous avons également choisi d'enregistrer les données dans des fichiers json, pour avoir l'opportunité si besoin de relancer qu'une partie des pipelines sans avoir à refaire les appels au service d'ingestion.

Une demande de récupération de données aura donc besoin des paramètres suivants:

- symbol de la crypto-monnaie
- période de récupération (chaque seconde, chaque heure ...),
- Nombre de points à récupérer
- Nombre de points à récupérer par page (dépend de l'échange)
- Chemin de sortie du fichier d'export

Routes

ingestion

POST	/ingestion/run	Run Ingestion
GET	/ingestion/stream/{coin_id}	Stream Candle

L'ingestion est géré à partir de deux routes :

- **POST /ingestion/run** : appel ccxt et enregistre le résultat des différentes pages dans un fichier (accepte les paramètres présentés dans la précédente partie)
- **GET /ingestion/stream/{coint_id}** : récupère en la dernière bougie de prête (en passant par le système de websocket de ccxt)

Coingecko

Coingecko est un site de trading pour les crypto-monnaies. Il affiche une liste exhaustive des crypto-monnaies existantes, et fournit une API permettant entre autres de récupérer cette liste.

L'intégration de coingecko dans notre projet a un seul but, fournir une liste de crypto-monnaies qu'il est possible de gérer, avec pour chacune l'échange à utiliser. Nous décrivons cette liste avec le terme "catalogue".

L'API Coingecko est appellable par des requêtes HTTP classiques, mais nous avons préféré utiliser le client python fourni par Coingecko.

Coingecko a besoin d'une clé API pour être utilisé. Nous stockons celle-ci dans un fichier `.env`, qui n'est pas versionné dans git. L'initialisation du client python se fait de cette façon :

```
load_dotenv(override=True)
COINGECKO_API_KEY = os.environ.get("COINGECKO_API_KEY")

client = Coingecko(
    demo_api_key=COINGECKO_API_KEY,
    environment="demo",
)
```

Pour mettre en place la correspondance entre les monnaies et le exchanges différentes opérations sont nécessaires :

- Récupération de la liste des crypto-monnaies : cette étape est faite en appelant la route **GET /coins/markets**. C'est fait de cette façon avec le client python

```
coin_markets = client.coins.markets.get(
    vs_currency='eur',
    per_page=length,
    page=1
)
```

- Récupération de la liste des exchanges : cette étape est faite en consultant la liste des exchanges fournis par ccxt, pour chacun on fera ensuite une autre requête pour avoir la liste des cryptomonnaies qu'il est capable de gérer. La liste des exchange se fait avec cette appel :

```
exchange_names = ccxt.exchanges
```

Ensuite pour chaque exchange il faut récupérer la liste des monnaies qu'il gère:

```
def get_markets_length(name):
    try:
        exchange = getattr(ccxt, name)()
        markets = list(exchange.load_markets().values())
        return {
            'name': name,
            'nb': len(markets),
            'markets': [{'base': x['base'], 'quote': x['quote']} for x in markets]
        }
    except Exception as e:
        return {
            'name': name,
            'nb': 0,
            'markets': []
        }
```

Chaque appel à l'API CCXT pour récupérer la liste des monnaies d'un échange prend un certain temps, il est donc plus intéressant de paralléliser cette partie pour gagner du temps. La récupération de ces informations sera alors faite dans un pool, de cette façon :

```
with Pool() as p:
    exchanges = p.map(get_markets_length, exchange_names)
exchanges.sort(key=lambda x: x['nb'], reverse=True)
```

La liste des échanges est ensuite triée par nombre de monnaies générées (on considère qu'un échange avec plus de monnaies qu'un autre sera considéré comme plus intéressant).

- Association entre les monnaies et les échanges : Cette étape est beaucoup plus simple. Dès que les appels API sont faits, il suffit de parcourir notre liste de monnaie, et pour chacune on parcourt la liste des exchange pour trouver le premier qui la gère.

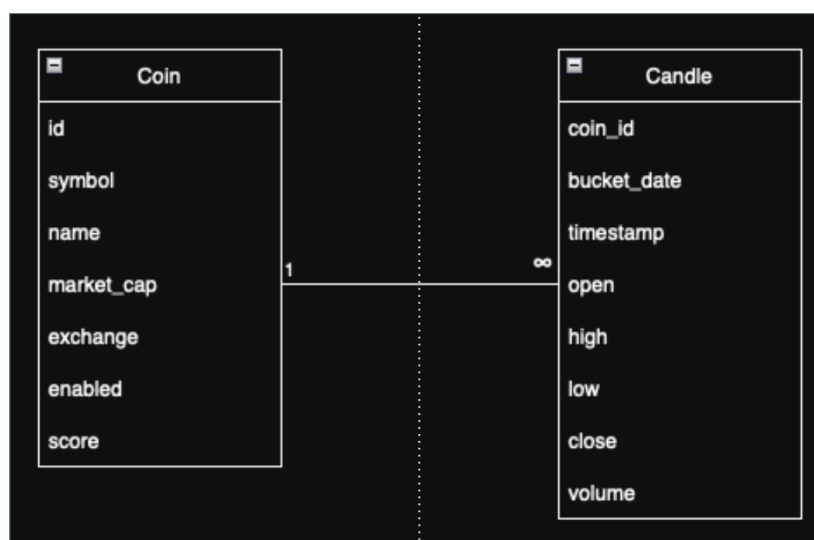
Au final nous souhaitons pouvoir gérer 250 cryptomonnaies (limite maximum d'une seule requête de **/coins/markets** coingecko). C'est un choix arbitraire, on pourrait très bien en gérer plus. Dans les faits nous ne prendrons en compte que 219. Certaines n'ont pas de correspondances vers un échange CCXT, nous avons choisis de les exclure. Dans le cadre de notre projet, cela nous donne une bonne base de travail.

Base(s) de données

Concernant le stockage des données de ce projet, il a été choisi d'utiliser deux types de base de données :

- Une base relationnelle classique pour le paramétrage, que nous appellerons base de données d'administration. Nous avons choisi PostgreSQL
- Une base de type colonne pour le stockage des cours des crypto-monnaies. Nous avons choisi Cassandra (qui était également disponible en tant que contenu facultatif de cours)

Le schémas de nos base de données reste très simple vis à vis des fonctionnalités que nous avons choisi d'implémenter :



A gauche on retrouve la table PostgreSQL de gestion du choix des crypto-monnaies à gérer, et à droite on retrouve la table Cassandra de stockage du cours des crypto-monnaies.

Il n'y a pas de connexion entre les deux bases à proprement parler, mais on peut virtuellement considérer une clé entre les deux tables d'un point de vue conceptuel. Le champ **coin_id** de la table Cassandra sera forcément une valeur du champ **id** de la table PostgreSQL.

PostgreSQL

La base de données PostgreSQL est initialisée grâce à ce script :

```
CREATE TABLE IF NOT EXISTS public.coin
(
    id text COLLATE pg_catalog."default" NOT NULL,
    symbol text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    market_cap bigint NOT NULL,
    exchange text COLLATE pg_catalog."default" NOT NULL,
    enabled boolean DEFAULT false,
    score float DEFAULT -1,
    CONSTRAINT coin_pkey PRIMARY KEY (id)
);
```

Elle est manipulable avec les routes api suivantes :

coin

GET	/coin/	List
GET	/coin/{id}	Item
PATCH	/coin/{id}/enable	Enable
PATCH	/coin/{id}/disable	Disable

- **GET /coin/** : liste les monnaies présentes en base
- **GET /coin/{id}** : récupère une monnaies présente en base, à partir de son id
- **PATCH /coin/{id}/enable** : active une monnaie pour être gérée par le pipeline (passage du champ enable à true)
- **PATCH /coin/{id}/disable** : désactive une monnaies pour ne plus être gérée par le pipeline (passage du champ enable à false)

Cassandra

Mettre en place une base de données Cassandra est pas aussi “immédiat” qu’une base de données relationnelles classique, plusieurs concepts importants sont à aborder.

La modélisation d’une base de données Cassandra doit se faire à partir des requêtes qu’elle est susceptible de recevoir. Pour cette raison nous sommes restés simples dans l’utilisation de notre base de données, uniquement destinée au stockage et à la lecture des bougies des crypto-monnaies.

Modélisation

La table des bougie sera initialisée grâce au script **cql** suivant :

```
CREATE KEYSPACE IF NOT EXISTS crypto_bot
WITH replication = { 'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE IF NOT EXISTS crypto_bot.candles (
    crypto_id text,
    bucket_date date,
    timestamp timestamp,
    open decimal,
    high decimal,
    low decimal,
    close decimal,
    volume decimal,
    PRIMARY KEY ((crypto_id, bucket_date), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Par défaut, une base Cassandra ne possède pas de keyspace (l'équivalent d'un namespace PostgreSQL). Il doit donc être créé dans ce même script.

Concernant la table candle, le script définit deux type de clé différentes, réunies conjointement dans la clé primaire :

- Clé de partitionnement : elle permet de regrouper certaines bougies dans une même partition Cassandra. Les bougies seront donc regroupées par leur ensemble **id + bucket_date**. (bucket_date étant un choix de date dépendant de la fréquence des points récupérés depuis CCXT, choix qui lui même dépendra des besoin de notre algorithme de machine learning)
- Clé de clustering, utilisée comme clé de tri lors des requêtes. Il s'agira du **timestamp** des bougies.

Test de performance

Avant de choisir le pas de la bucket_date, un comparatif de performance a été fait entre Cassandra et PostgreSQL (si jamais on l'aurait utilisé à la place pour stocker les bougies), pour estimer si utiliser Cassandra apporterait un vrai avantage à la lecture/écriture, même si il en a pour d'autres contextes tels que le streaming et le calcul distribué.

Un premier test a été fait pour des bougies récupérées toutes les secondes, mais il n'a pas été poussé loin car il donnait de mauvaises performances pour le machine learning. Cependant il avait été observé qu'à la lecture PostgreSQL et Cassandra avaient des performances équivalentes pour stocker ou récupérer 50000 points en une fois. Ce chiffre est devenu une valeur de référence concernant la taille des jeux de données que nous traiterons à différents endroits du projet.

Le pas en secondes ayant été exclu , il a resté un comparatif à faire entre un choix par minute ou par heure

Récupération par minute (partitionnement de Cassandra par mois) :

	PostgreSQL	Cassandra
lecture	0.08	0.42
écriture	1.49	1.82

Récupération par heure (partitionnement de Cassandra par an) :

	PostgreSQL	Cassandra
lecture	0.09	0.43
écriture	1.46	1.63

Les temps sont présentés en secondes. On peut remarquer que le choix d'une période de récupération des bougies en minutes ou en heure n'a pas grande influence sur les performances générales. De plus, PostgreSQL semble plus performant en lecture pour l'instant. Cassandra lit toujours en 4 fois plus de temps, même si le temps final reste acceptable pour un jeu de données de 50 000 points.

Cependant Cassandra a été testé dans un cadre de projet local, avec un facteur de réplication de 1 pour implémenter une première configuration simple. Une autre analyse devra être envisagée pour une version en production de Cassandra.

Partitionnement

Vis à vis des performances du machine learning (sujet qui sera abordé plus loin), on va préférer récupérer des bougies à une fréquence de une heure. Concernant le partitionnement de Cassandra, ces exemples bonnes pratiques peuvent être à prendre en compte

- Moins de 100.000 éléments par partition
- Taille de partition inférieur à 100 Mo
- Taille d'une ligne inférieur à 10MB

Dans notre cas, si nous choisissons un pas récupération d'une heure, et un partitionnement à l'année, nous pouvons faire le calcul approximatif suivant :

- Taille par ligne : **16** (coin_id) + **10** (bucket_date) + **64** (timestamp) + **8** (open) + **8** (high) + **8** (low) + **8** (close) + **8** (volume) = **130** octets par ligne
- **8784** lignes (366*24) (chaque partition aura la taille d'une année bisextile)
- **1,1 Mo** par partition (8784 * 130)

Dans notre cas, cela confirme que nous ne dépassons pas les bonnes pratiques présentées plus haut.

Manipulation

Une opération générique sur une table Cassandra (récupérer, supprimer ... un ensemble de données), ne se fait pas avec une simple requete **SELECT** comme pour une base de données relationnelle classique.

Pour chacune de ses opérations à effectuer, il va falloir calculer les différentes "bucket date", et faire une requête pour chaque partition correspondante. Prenons l'exemple du code pour la suppression des données du crypto-monnaie :

```
def remove_cassandra_candles(coin_id, session: Session):
    query = """
        DELETE FROM candles
        WHERE crypto_id = ? AND bucket_date = ?
    """
    prepared = session.prepare(query)

    first_year = 2009
    current_year = datetime.now(timezone.utc).year
    nb_year = current_year - first_year + 1

    futures = []
    current_date = datetime.now(timezone.utc)
    bucket_date = current_date.replace(second=0, microsecond=0).date()
    for i in range(nb_year):
        target_bucket = (datetime.combine(current_date, time()) - relativedelta(years=i)).date()
        future = session.execute_async(prepared, (coin_id, target_bucket))
        futures.append(future)

    for future in futures:
        future.result()
```

Une manipulation de la table candle se fera alors en plusieurs étapes :

- Définition d'une requête préparée, appellable plusieurs fois avec des paramètres différents
- Détermination du nombre de partitions à requêter (par convention on considère 2009 comme année minimale dans le passé, année de création du bitcoin, la toute première crypto monnaie)
- Appel de la requête pour chaque année, en utilisant des exécution asynchrone pour optimiser le temps global
- Attente de la fin de l'exécution des différentes requêtes asynchrones

Routes

Pour finir nous aurons donc besoin des routes suivantes dans FastAPI :

candle



GET	/candle/{id}/list	List	✓
POST	/candle/{id}/save	Save	 ✓
GET	/candle/{coin_id}/interval	Last Date	✓
DELETE	/candle/{id}/remove	Remove	✓

- **GET /candle/{id}/list** : liste les bougies d'une crypto-monnaie donnée.
- **POST /candle/{id}/save** : enregistre les bougies d'une crypto-monnaie donnée.
- **GET /candle/{coin_id}/interval** : récupère les dates de la première et de la dernière bougie pour une crypto-monnaie donnée (informations nécessaires aux pipelines airflow).
- **DELETE /candle/{id}/remove** : supprime les bougies d'une crypto-monnaie donnée.

Machine learning

Difficultés rencontrées

La mise en place du machine learning dans ce projet a présenté plusieurs difficultés. Premièrement, gérer des données temporelles a été un exercice différent de ce que nous avons pu voir en cours, et a demandé de mettre en place une approche plus spécifique.

De plus, il a fallu définir ce qu'est la notion de prédiction dans le cadre de notre projet. Faire de la prédiction dans un projet de crypto-monnaies peut vouloir dire plusieurs choses (est-ce qu'on prédit directement l'évolution du cours, est-ce qu'on prédit la bougie entière, jusqu'à quand on prédit dans le futur ?...)

Et contrairement à d'autres projets, il n'y pas de solution clé en main pour prédire l'évolution de la crypto-monnaie. Prévoir le cours directement nous a pas donné de bon résultat, prédire la hausse ou la baisse quantitativement non plus. Nous avons donc choisi de nous mettre dans le cas le plus simple possible, puis de prévoir de le faire évoluer petit à petit.

Prédiction par classification

Nous avons donc choisis de partir sur le test de différents modèles de machine learning de type classification, pour faire des prédictions simples : 0 ou 1. Ces deux valeurs auront la signification suivante :

- 0 représente la prédiction d'une baisse. D'un point de vue utilisateur, on affichera une recommandation de vente.
- 1 représente la prédiction d'une hausse. D'un point de vue utilisateur, on affichera une recommandation de vente.

Nos premiers tests se sont portés sur le cours de la monnaie “Bitcoin”. Nous avons envisagé les différents modèles facilement accessible avec la librairie sklearn :

Modèle	Score
RandomForestClassifier	0.59
GradientBoostingClassifier	0.52
LogisticRegression	0.55
DecisionTreeClassifier	0.54

Même si les écarts entre les différents résultats ne sont pas énormes, nous avons gardé le modèle **RandomForestClassifier** comme modèle de notre pipeline de machine learning.

Concernant la métrique utilisée en tant que score, nous avons choisi la précision, même si nous aurions pu utiliser le f1-score tel que vu en cours, pour intégrer en plus le rappel. Notre choix a été guidé avec la logique suivante (en gardant en tête l’approche applicative de notre projet) :

- Dans le cas d’un faux négatif, cela veut dire qu’on prédit une baisse au lieu d’une hausse. Donc on va indiquer à l’utilisateur de vendre au lieu d’acheter. On considère que ce n’est pas trop grave, puisque si on se place dans le cadre d’une application de trading, l’utilisateur sait combien il a acheté une monnaie, donc il sait si le cours actuel ne lui permet pas d’avoir de gains.
- Dans le cas d’un faux positif, on va prédire une hausse, et donc indiquer à l’utilisateur qu’il vaut mieux acheter maintenant. Sauf que si l’utilisateur achète, il doit être sûr d’avoir des opportunités pour revendre. c’est pour cette raison qu’on privilégie la **précision** en tant que métrique de comparaison entre différentes versions de modèle, métrique censé minimiser les faux positifs.

Transformation des données

L'entraînement de notre modèle de machine learning à nécessité la transformation des données bougie initiales, afin de rajouter des informations supplémentaires, améliorant les performances de nos prédictions.

Voici les différentes étape qui sont faite lors de cette phase de transformation (à partir de la liste des bougies) :

- Lissage des valeurs de la colonne **close**, colonne qui sert de base pour déterminer la donnée cible de notre modèle. Ce lissage se justifie par le fait que la courbe du cours d'une crypto-monnaie possède beaucoup de bruit, même si on peut en observer des tendances.
- Création de la donnée cible à partir de cette donnée lissée, en prenant le 24ème point dans le futur (pour réaliser des prédictions 1 jour dans le futur), puis en déterminant si il est en hausse ou baisse par rapport au point actuel.
- Ajout d'informations pour que le modèle ait des informations du passé. Pour chaque point on a décidé d'ajouter les 10 valeur cibles du passé.

Le modèle travaillera donc avec :

- données **close, open, high, low, volume**, hausse/baisse des **10** point dans le passé (ce qui fait **15** features au total)
- état de la hausse ou baisse entre le jour suivant et le jour actuel en tant que donnée cible.

Routes FastAPI

ml		^
GET	/ml/transform/{coin_id} Transform	▼
POST	/ml/train Train	▼
POST	/ml/predict Predict	▼

Notre API fournit deux routes pour manipuler le machine learning :

- **GET /ml/transform/{coin_id}** , transforme les données de bougies pour une monnaie donnée (à partir des données déjà présentes en base)
- **POST /ml/train** , entraînement du modèle pour une monnaie donnée (en sauvegardant le modèle si le score est meilleur que le précédent)
- **POST /ml/predict** , exécute une prévision pour une monnaie donnée (en sauvegarde du modèle si le score est meilleur que le précédent). (Cette en interne utilise le mécanisme de websocket de CCXT pour prédire à partir du point le plus récent.

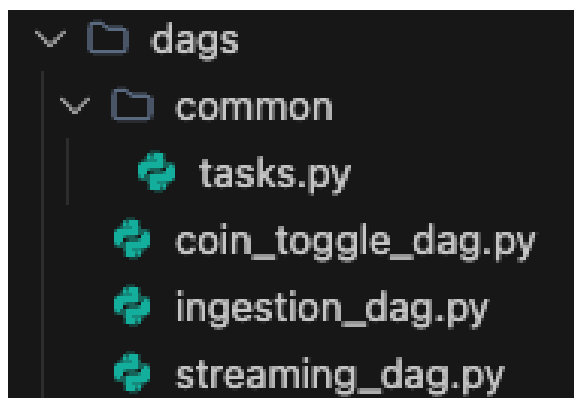
Pipeline Airflow

Nous avons choisi Airflow pour récupérer et traiter de façon automatique les données des différentes crypto-monnaies. La seule connexion qu'aura Airflow vers le reste du projet se fera à travers notre API FastAPI.

Dans les pipelines que nous avons mis en place, on peut distinguer deux types de DAG :

- Manuels : doit être appelé depuis l'API
- Périodiques : est exécuté à une période donnée (chaque minute, chaque seconde ...)

Nos fichiers de DAG sont structurés de la façon suivante :



- **coin_toggle_dag** : doit être exécuté manuellement à chaque activation/désactivation d'une monnaie depuis le dashboard streamlit d'admin
- **ingestion_dag** : gestion complète du pipeline, exécuté chaque moi
- **streaming_dag** : gestion du streaming, exécuté chaque heure pour enregistrer en base la dernière bougie disponible
- **common/tasks** : contient les parties communes des différents DAGs (tâche ; task-groups ...)

Choix techniques

Nous avons voulu factoriser et optimiser la structure du code de nos DAGs, en utilisant au mieux les fonctionnalités de Airflow que nous avons à disposition.

Dynamic Task Mapping

Dans la construction d'un DAG airflow, il peut être parfois nécessaire plusieurs tâches en parallèle, sans connaître à l'avance le nombre de ces tâches.

Dans le cadre de notre projet, il s'agit de pouvoir être capable d'effectuer nos pipeline sur l'ensemble des crypto-monnaies sélectionnés dans notre dashboard (si le pipeline concerné a besoin de cette donnée).

Pour pouvoir gérer cet aspect dans Airflow , nous avons utilisé le "Dynamic Task Mapping", qui permet de répondre à ce besoin. En terme de code, cela se fait par exemple de cette façon :

```
@task_group(group_id="main_pipeline")
def main_pipeline(coin_id_list: List[str]):
    t2 = process_single_coin.expand(coin_id=coin_id_list)
```

Dans cet exemple, la tâche **process_single_coin** sera dupliquée autant de fois que de monnaies données en paramètre. Ensuite chaque tâche qui suivra cette première devra être déclarée avec **.expand**, pour que la division du tableau paramètre se poursuive.

Task group

En termes de bonne pratique, dès qu'un ensemble de tâche devra être utilisé dans différents DAGs (ou dans plusieurs endroits d'un même DAG), il sera regroupé dans un "task group". Un task group se déclare simplement de cette façon :

```
@task_group(group_id="main_pipeline")
def main_pipeline(coin_id_list: List[str]):
    t2 = process_single_coin.expand(coin_id=coin_id_list)
```

Duplication de tâches

Il peut parfois être possible d'utiliser deux fois la même tâche, par exemple dans deux branchements différents d'un même DAG. Dans ce cas de figures, pour ne pas avoir de conflits dans les IDs de ces tâches, on les invoquera en utilisant en plus la méthode `override`, qui permet de dupliquer une tâche en lui indiquant un nouvel ID. Par exemple :

```
t5 = extract.override(task_id=f"{prefix}_extract").expand(coin_data=coin_data_list)
```

Appel manuel

Suivant le cas on peut avoir besoin d'appeler manuellement un DAG depuis le codé de notre API. Voici un exemple de code répondant à cette problématique :

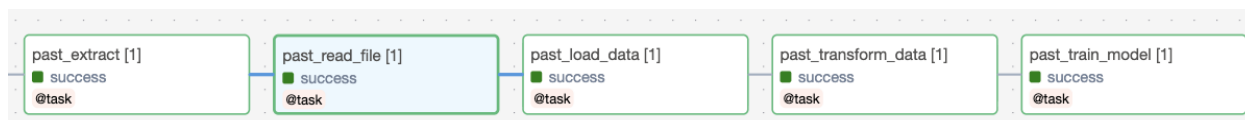
```
httpx.post("http://airflow-webserver:8080/api/v1/dags/cryptobot_coin_toggle/dagRuns",
    json={
        "conf": {
            "command": "enable",
            "coin_id": id
        },
    },
    auth=("airflow", "airflow"))
```

(l'id indiqué dans l'URL est celui défini dans l'annotation `@dag` d'un pipeline Airflow)

DAGs

Task groups

Différents task groups ont été définis pour le bon fonctionnement de nos pipelines. Le premier à présenter est celui responsable de l'aspect ETL de notre pipeline (En réalité nous appliqueront plutôt une approche ELT. Comme l'étape de machine learning n'est pas définitive, l'étape de transformation des données se déroule après l'étape de chargement des données en base. A partir des données bougies en base, nous les transformant en données requises par le machine learning)



Les tâches de ce task group ont un préfix "past_" dans leur nom pour illustrer la branche qui récupère les données dans le passé. Ce task group est constitué des tâches suivantes :

- **extract** : lancement de l'ingestion pour la monnaie courante
- **read_file** : lecture des données du fichier généré par l'ingestion
- **load_data** : charge les données en base
- **transform_data** : transforme les données pour les fournir au machine learning
- **train_model** : entraîne le modèle pour la monnaie courante

Le deuxième task group à présenter est le pipeline principal, pouvant être appelé à deux moments, quand on active une monnaie, et régulièrement une fois par mois.

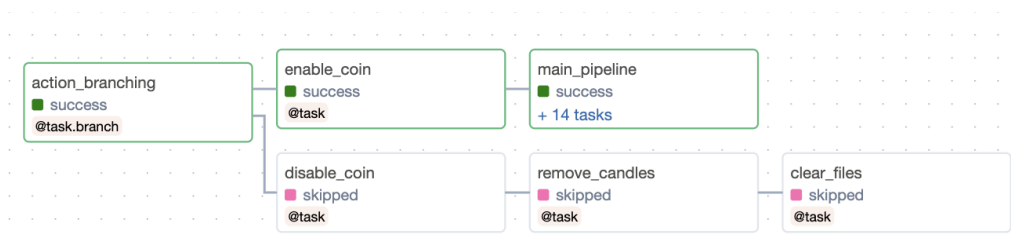


Il est constitué des tâches suivantes :

- **process_single_coin** : transmet l'identifiant de la monnaie courante à la tâche suivante.
- **get_interval** : récupère les dates des premières et dernières bougies présentes en base. Si elle ne sont pas définies, on va récupérer les données du passé, sinon on va récupérer les données du présent (entre la date la plus récente en base et le début du mois du DAG)
- **past_*/present_*** : appel du task group ELT présenté précédemment

coin_toggle_dag

Ce DAG est responsable de réagir aux activations et désactivations d'une monnaie depuis le dashboard. Il doit être appelé manuellement depuis notre API.



Il est constitué des étapes suivantes :

- **action branching** : détermine l'action suivante en fonction du paramètre reçu dans la requête
- activation d'une monnaie:
 - **enable_coin** : récupère l'id de monnaie fournie en paramètre
 - **main_pipeline** : appelle le pipeline principal
- désactivation d'une monnaie:
 - **disable_coin** : récupère l'id de monnaie fournie en paramètre
 - **remove_candles** : supprimer les bougies présente en base pour la monnaie concernée
 - **clear_files** : supprime les fichiers générés pour la monnaie donnée

ingestion_dag

Ce pipeline est appelé tous les mois pour mettre à jour les modèles de machine learning



. Il réalise les étapes suivantes :

- **pull_catalog** : récupère les monnaies activées, puis les dispatch en autant de tâches "main_pipeline"
- **main_pipeline** : appelle le pipeline principal (présenté précédement)

streaming_dag

Ce dernier pipeline est exécuté chaque heure.



Plus simple que les autres, il est responsable des étapes suivantes:

- **pull_catalog** : récupérer la liste des monnaies activées
- **request_candle** : récupère la bougie la plus récente que peut nous fournir la websocket de ccxt
- **load_data** : charge cette données en base

Tests unitaires

Nos tests unitaires se contentent de tester certaines routes de notre API, notamment pour vérifier la cohérence de leurs retours, par exemple pour avoir le bon code d'erreur si les paramètres sont mal renseignés. Ils peuvent se lancer de deux manières différentes :

- localement avec la commande : **docker compose run --rm tests**
- automatiquement notre pipeline CI/CD (avec Github actions)

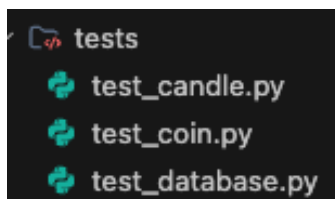
Exemple de test unitaire :

```
import httpx

def test_root():
    url = "http://api:8000/"
    response = httpx.get(url)
    assert response.status_code == 404
```

(Il n'y a pas de route associée à la racine de notre API, on doit donc bien avoir une erreur 404 en réponse)

Structure de notre dossier de test:



On y retrouve les fichiers suivants:

- **test_candle** : test certaines routes de manipulation des bougies :
 - récupération de bougies (route **GET /candle/bitcoin/list**)
 - suppression de bougies (route **DELETE /candle/bitcoin/remove**)
- **test_coin** : test certaines routes de manipulations des monnaies :
 - liste des monnaies (route **GET /coin**)
 - génération de la liste des monnaies (route **GET /admin/postgres/fill**)
- **test_coin** : test certaines routes d'accès aux base de données :
 - base de données postgresSQL initialisé (route **GET /coin**)
 - base de données cassandra initialisée (route **GET /candle/bitcoin/list**)

Monitoring

La mise en place de notre projet s'est principalement concentré sur le développement local. Nous n'avons pas testé une vraie mise en production, mais nous avons une structure docker complète pour pouvoir le faire plus tard.

Pour cette raison la mise en place de dashboard de monitoring avec prometheus et grafana s'est principalement porté sur l'aide au débogage.

FastAPI

Pour monitorer FastAPI, nous avons intégré la librairie `prometheus_fastapi_instrumentator`, puis utilisé le dashboard existant **FastAPI Observability** dans grafana, ce qui donne ce résultat :



Logs

Nous avons également choisi de mettre en place un dashboard pour avoir les logs de chaque service de docker. Ainsi on est plus obligés de lire les différents logs dans chaque terminal.

The screenshot displays a dashboard with four panels, each showing logs for a different service. The panels are titled 'Logs - Service FastAPI', 'Logs - Service PostgreSQL', 'Logs - Service Airflow', and 'Logs - Service Cassandra'.

- Logs - Service FastAPI:** Shows several log entries. The first three are HTTP requests: a GET to /metrics (200 OK), a GET to /docs (200 OK), and a POST to /ml/train (204 No Content). The next two entries are from sqlalchemy.engine.Engine, showing a ROLLBACK and a successful query execution (cached since 66.39s ago).
- Logs - Service PostgreSQL:** Shows two log entries. Both are INFO messages from the LOG: checkpoint complete. The first entry is dated 2026-07-21 17:28:20.131 UTC and shows 126 buffers written. The second entry is dated 2026-07-21 17:28:08.336 UTC and shows 2 buffers written.
- Logs - Service Airflow:** Shows three log entries. The first two are INFO messages from INFO/ForkPoolWorker-15, showing task execution and running status. The third entry is an INFO message from INFO/ForkPoolWorker-15, showing the DAGBag being picked up.
- Logs - Service Cassandra:** The panel is empty, displaying 'No data'.

Nous avons choisi de monitorer les services les plus utiles à la résolution des problèmes que nous avons pu rencontrer :

- FastAPI
- PostgreSQL
- Airflow
- Cassandra

CI/CD

Les sources de notre projet étant hébergé sur github, nous avons choisi d'utiliser les Github actions pour nos pipelines CI/CD. Nous en avons mis deux en place (stockés dans `.github/workflows`) :

Test unitaire

```
name: Lancement des tests unitaires

on:
  workflow_call:
  push:
    branches:
      - main

jobs:
  run-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout on main
        uses: actions/checkout@v4

      - name: Build API service
        run: docker compose build api

      - name: Run tests
        run: docker compose run --rm tests

      - name: Teardown containers
        if: always()
        run: docker compose down -v
```

Ce pipeline est lancé à chaque commit sur main, pour être sûr qu'un minimum d'erreurs ne soit présent. Si tout se passe ça donne le résultat suivant :

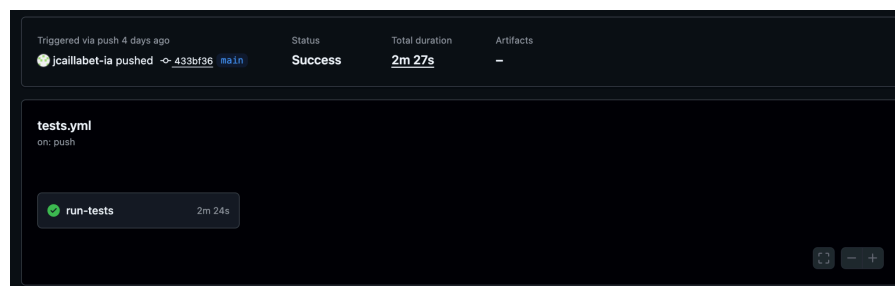


Image docker

```

name: Déploiement de l'image FastAPI sur docker hub

on:
  push:
    tags:
      - 'v*'

jobs:
  call-tests-workflow:
    uses: ../github/workflows/tests.yml

  build-and-push:
    runs-on: ubuntu-latest
    needs: call-tests-workflow

    steps:
      - name: Checkout on main
        uses: actions/checkout@v4

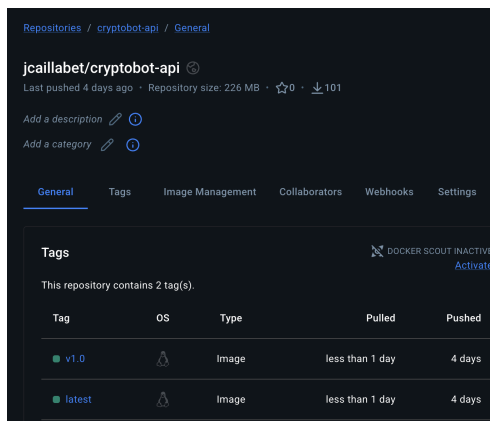
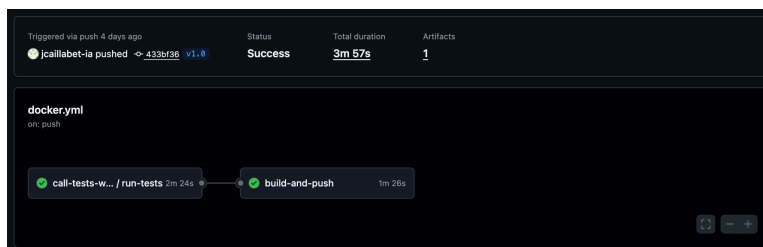
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Build and push Docker image
        uses: docker/build-push-action@v6
        with:
          context: ./api
          file: ./api/Dockerfile
          push: true
          tags: |
            ${ secrets.DOCKERHUB_USERNAME }/cryptobot-api:latest
            ${ secrets.DOCKERHUB_USERNAME }/cryptobot-api:${ github.ref_name }

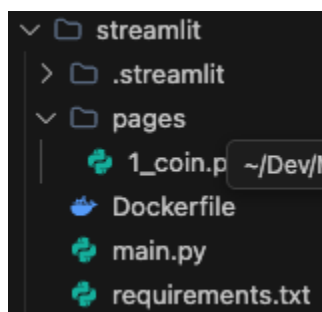
```

Ce pipeline est chargé de build une image docker de notre API, puis de la mettre en ligne sur docker hub. Il vérifie également les tests unitaires avant. Si tout se passe bien il donne le résultat suivant :



Dashboards

Les dashboard de démonstration de notre projet ont été envisagés avec streamlit. Dans notre projet il est paramétré comme un simple conteneur python, avec l'arborescence suivante :



Le fichier main.py contiendra le code de notre dashboard principal, et le dossier pages contiendra tous les autres.

Si on reprend la philosophie présentée en introductions, deux dashboard ont été implémentés avec streamlit.

Administration

Crypto-bot dashboard				
id	symbol	Action	Score	Consulter
bitcoin	BTC/USDT	Désactiver	0.5847	Consulter
ethereum	ETH/BTC	Désactiver	0.4992	Consulter
tether	USDT/TRY	Activer	0	
binancecoin	BNB/BTC	Activer	0	
usd-coin	USDC/BNB	Activer	0	
ripple	XRP/BTC	Activer	0	
solana	USDSOLD/USDT	Activer	0	
tron	TRX/BTC	Activer	0	
hyperliquid	HYPER/USDT	Activer	0	
dogecoin	DOGE/BNB	Désactiver	0	Consulter

Cet écran nous permet d'activer ou désactiver une crypto-monnaie de notre pipeline. On peut aussi voir pour chaque monnaie le score actuel du modèle de prédiction, et d'avoir accès à l'écran de consultation de chaque monnaie activée.

Consultation



Cet écran nous permet de consulter le graphique d'évolution du cours d'une crypto monnaie, et d'avoir la prédiction actuelle (hausse ou baisse)

Conclusion / Piste d'amélioration

Même si notre projet est fonctionnel par rapport aux fonctionnalités que nous voulions mettre en place, il y a bien sur plusieurs pistes d'évolutions à imaginer pour une version future :

- **Meilleure utilisation de Cassandra** : pour l'instant nous nous sommes contentés d'une utilisation locale de Cassandra, mais avec une vraie mise en production et une vraie utilisation du mode distribué nous pourrions avoir de meilleures performances en lecture, notamment pour le streaming.
- Réussir des prédictions exploitables avec des **pas de données à la seconde** : Cela permettrait un véritable affichage temps réel, à la fois du cours et de la prédictions.
- Intégrer d'autres **sources de données** au machine learning : Par exemple, traiter des articles liés au monde de la crypto-monnaies avec de l'analyse de sentiment, permettrait de faire des prédictions avec en plus des événements futurs.
- **refactoring** de certains aspects du projet. Par exemple définir à un seul endroit le pas des données, ou la construction des clés Cassandra, permettrait de tester plus facilement d'autres configurations possibles.

Sources

(Quelques sources pour justifier certains de nos choix)

<https://fastapi.tiangolo.com/advanced/events/#async-context-manager>

<https://dba.stackexchange.com/questions/330705/is-it-okay-to-have-1000-rows-in-a-single-partition>